

A Data-Driven Football Predictions API

COMP3011 Coursework 1 – API Development

Technical Report

Project Introduction

This project focuses on the design and development of a data-driven web API that manages information related to each team's probability of winning the UEFA Europa League. The system stores structured records for every participating team, capturing their current win probability along with relevant contextual information. The aim of this model is to demonstrate how structured sports data can be analysed to estimate tournament outcomes. Additionally, the system has been designed to be adaptable, meaning that data from other football competitions could be incorporated and the same predictive model applied to estimate potential winners in different tournaments.

The system supports multiple interconnected data models, enabling full CRUD functionality for teams, players, and matches, each stored within a structured and persistent database. This ensures that the core football information required for analysis is accurately maintained and can be modified or retrieved as needed. The overall design follows a RESTful architecture, exposing clearly defined HTTP endpoints and returning structured responses in a standard format. The system is modular, extensible, and intended to serve as a foundation for more advanced analytical capabilities and modelling.

A key motivation for choosing this topic is my genuine interest in the competition, which makes the Europa League an engaging and meaningful setting for a technical project. I was also inspired by a sports analytics discovery module I completed in my second year, where I was introduced to concepts such as statistical modelling, performance metrics, and probability-based analysis in football. This project gives me the opportunity to apply that knowledge in practice and develop it further within a real, system-focused context. By combining my enthusiasm for the competition with the analytical skills gained during my studies, the project becomes both academically valuable and personally rewarding.

Design

Requirements

The requirements outline what the system must achieve to effectively manage and analyse Europa League football data. They define both the essential functionality of the API and the quality standards it must meet, forming the basis for the system's overall design.

Functional Requirements

- The system must manage structured data related to teams, players, and matches within the context of the UEFA Europa League.
- The API must provide full CRUD functionality for all core data models.
- Endpoints must allow clients to create, retrieve, update, and delete records for teams, players, and matches.
- The system must support analytical operations, including generating leaderboards, performance summaries, and win-probability outputs.
- All endpoints must return structured responses in a consistent format, such as JSON.

- The API must validate incoming data to ensure correctness and integrity before processing.

Non-Functional Requirements

- The system must follow RESTful design principles, using predictable and standardised endpoint structures.
- HTTP methods and status codes must be used appropriately to reflect the outcome of each operation.
- The API should maintain reliable performance when handling multiple requests or larger datasets
- The database must ensure data consistency, integrity, and support relationship mapping between models.
- The architecture should be modular and extensible, allowing for additional analytical features or data models to be added in the future.
- The system should be straightforward to deploy and demonstrate, working reliably in both local and hosted environments.

Use Cases

Within the appendix of this report, a table is provided outlining the key use cases that the system must support to meet the project requirements and ensure that all core functionality is fully achievable.

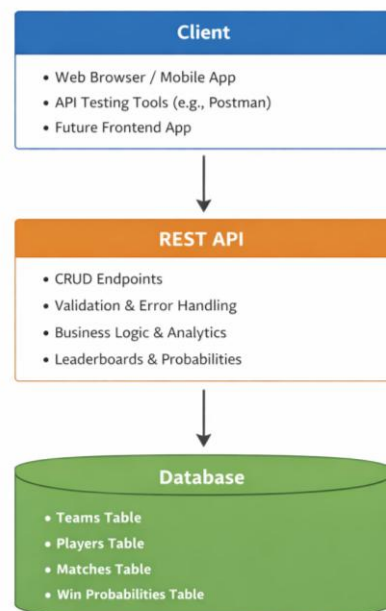
System Design

High Level Architecture

At its core, the architecture follows a client-server model in which external clients interact with the API through well-defined HTTP endpoints.

The API layer handles request processing, validation and business logic, while a dedicated database layer is responsible for storing and organising data on teams, players, matches, and analytical outputs such as leaderboards and win probabilities.

This layered approach ensures loose coupling between components, supports scalability, and provides a modular foundation that can be extended with additional functionalities or analytical models as the system evolves.



Data Model

The data model defines the structure of the system's core entities and outlines how football data is stored, linked, and used for analysis within the context of the UEFA Europa League. The model is built around four main entities: Teams, Players, Matches, and Probabilities, each serving a specific purpose in managing competition data.

Teams form the central entity, with one-to-many relationships to both Players and Matches. Players are linked to a single team through a foreign key, while Matches reference two teams, one as the home team and one as the away team. Probabilities also connect to Teams, allowing probability records to be stored for different seasons or update points.

Win probabilities can be calculated using external data sources, such as bookmaker odds, form statistics, rating systems, or predictive models. Once calculated, these values can be stored in the Probabilities table, enabling the system to incorporate real-world analytical insights and update them as new information becomes available.

This structured, relational design ensures data consistency, supports efficient CRUD operations, and provides a strong foundation for additional analytics. Full table definitions for all entities are included in the Appendix.

API design

The API is designed using a RESTful, resource-oriented architecture that provides a clear and consistent way to access and manipulate the system's core data. Each major entity (Teams, Players, Matches, and Win Probabilities) is exposed as a resource with standard operations that map directly to HTTP methods. This ensures intuitive interaction patterns: creating new data through POST requests, retrieving data through GET, updating records with PUT or PATCH, and removing entries using DELETE.

To complement the core CRUD functionality, the API also includes a set of analytical endpoints. These are designed to compute derived outputs such as leaderboards, performance summaries, and updated win probabilities based on the stored match and team data. By separating analytical endpoints from data-management endpoints, the design maintains modularity and supports future expansion, such as adding more advanced metrics or integrating predictive models.

All endpoints are designed to return structured data in a consistent format, enabling predictable interactions for client applications. Standardised HTTP status codes are used to communicate success, validation errors, missing resources, and conflict conditions. The design also incorporates input validation and clear error messaging to ensure that the system behaves reliably under a range of valid and invalid client requests.

This high-level structure allows the API to serve as both a reliable data store and a flexible analytical layer, laying the groundwork for further functionality to be introduced in later development stages. Details of the API implementation and Endpoint definitions are explained later in the report.

Technology Selection and Rationale

Python was chosen as the primary programming language due to its clarity, flexibility, and strong ecosystem for working with data. Its readable syntax makes it easy to implement and maintain the functionality required for this project, including CRUD operations, validation logic, and analytical features. Python also offers excellent support for future development, such as integrating statistical models or data processing techniques, which aligns well with the analytical goals of the system.

FastAPI was selected as the web framework because it provides a modern and structured approach to building RESTful APIs. Its built-in request validation, automatic documentation, and clean separation of concerns make it particularly well suited for a project that involves multiple data models and analytical endpoints. Compared with potential alternatives, FastAPI offered the strongest balance of usability and performance. Flask was considered but would have required additional configuration for validation and documentation, while Django was deemed too heavy for an API-focused application that does not require a full web framework with templating and authentication built in.

SQLite was chosen as the database solution because it is lightweight, easy to configure, and ideal for local development in a coursework setting. It provides full relational capabilities without the overhead of running a separate database server, making it simple to deploy, test, and demonstrate. Alternatives such as PostgreSQL or MySQL were considered for their scalability and advanced features, but they were ultimately unnecessary for the scope of this project and would have introduced additional setup complexity. SQLite therefore provides the most practical balance between functionality, simplicity, and portability while still allowing the data model to be migrated to a more scalable SQL system in the future if required.

Implementation

API Details

The API is implemented as a RESTful service using FastAPI, which provides automatic documentation generation through Swagger UI. The API exposes endpoints for managing football data, generating predictions, and running tournament simulations. Full details of all available endpoints and their respective functions are available in the API Documentation referenced in the README file.

Testing and Verification

Testing was carried out at multiple stages of the development process to ensure the system functioned correctly and met the specified requirements. Initial testing focused on verifying that each API endpoint behaved as expected. FastAPI's built-in Swagger interface (/docs) was used to manually test CRUD operations for teams, players, matches, and team form data. Data ingestion was verified by running the Excel import script and confirming that records were correctly stored in the database. This included validating that foreign key relationships between teams, players, and matches were maintained. Prediction model verification involved checking that the Poisson distribution generated realistic scorelines and that the calculated probabilities for home win,

draw, and away win summed to approximately one. Edge cases, such as missing team form data, were also handled through validation checks within the service layer. The tournament simulation functionality was tested by running multiple bracket simulations and confirming that winners progressed correctly through each round of the knockout stage. Repeated simulations were also used to verify that the Monte Carlo method produced sensible probability distributions for potential tournament winners. Overall system verification confirmed that the API successfully supported data ingestion, prediction generation, and tournament simulation as intended.

Deployment

The API was deployed locally using Uvicorn, an ASGI server designed for high-performance asynchronous Python applications. To run the system, the following steps are required:

- Install project dependencies using the requirements.txt file.
- Start the FastAPI application using the Uvicorn server.
- Import the dataset from the Excel workbook.
- Access the API documentation through Swagger UI.

The current deployment uses a lightweight SQLite database for simplicity during development.

Project Summary

Evaluation

The development of the Data-Driven Football Predictions API was carried out with the assistance of generative AI tools, which supported the implementation process while maintaining the original design and requirements specified earlier in the report. By using the defined use cases, functional requirements, and data model as a structured foundation, the system was implemented efficiently while maintaining a clear and modular architecture.

The final system successfully meets the core objectives outlined in the requirements section. The API provides structured endpoints for managing teams, players, matches, and team form statistics while supporting analytical operations such as match predictions and tournament simulations. The use of FastAPI enabled rapid development and automatic documentation generation, allowing the system to be easily tested and interacted with through the Swagger interface.

The implemented Poisson-based prediction model provides a reasonable statistical approach to estimating match outcomes based on recent performance metrics. While the model is intentionally simplified, it demonstrates how probabilistic modelling can be integrated into an API-based analytics platform. Additionally, the tournament simulation functionality allows the system to estimate potential outcomes for the remaining stages of the competition.

One limitation of the current implementation is that the dataset is manually imported from an Excel workbook rather than automatically sourced from external football data providers. While this approach simplified the implementation and ensured reliable test data, future versions could benefit from automated data ingestion pipelines.

Overall, the project successfully demonstrates how a modular API architecture can support data-driven sports analytics while remaining extensible for future improvements.

Future Developments

Several improvements could be made to extend the functionality and realism of the system. One potential enhancement would be the integration of automated data collection from external football data providers or APIs. This would remove the need for manual data entry through Excel and allow the system to continuously update team statistics and match results.

Another improvement would involve expanding the prediction model. While the current Poisson-based model provides a simple statistical foundation, more advanced approaches such as Elo ratings, expected goals (xG), or machine learning models could improve prediction accuracy by incorporating additional performance metrics.

From a technical perspective, the project could also benefit from further production-oriented features, such as containerisation using Docker, deployment to cloud platforms, and the addition of automated unit and integration tests.

These enhancements would increase both the analytical capabilities and real-world applicability of the system.

Conclusion

This project successfully demonstrates the design and implementation of a data-driven football prediction system built using a RESTful API architecture. By combining structured football data with statistical modelling techniques, the system provides a platform capable of generating match predictions and simulating tournament outcomes.

The use of FastAPI and a modular architecture enabled the project to be developed efficiently while maintaining a clear separation of responsibilities between data models, business logic, and API endpoints. The system supports both data management and analytical operations, illustrating how sports data can be processed and analysed within a scalable web service.

Although the current implementation focuses on a simplified prediction model and manually imported data, the architecture provides a strong foundation for future enhancements. With additional data sources, more sophisticated predictive models, and improved deployment infrastructure, the system could be extended into a more comprehensive sports analytics platform.

Overall, the project demonstrates the practical application of API development, database design, and statistical modelling within the context of football analytics.

Appendix

Use Cases Table

Use Case ID	Name	Actor	Description	Preconditions	Success Outcome
UC1	Create Team	Analyst / Admin	Add a new team record for the UEFA Europa League.	Team does not already exist.	Team is created and assigned a unique ID.
UC2	View Teams	Analyst / Client App	Retrieve a list of all teams or details for an individual team.	At least one team exists.	Team data is returned in structured format.
UC3	Update Team	Analyst / Admin	Modify an existing team's information.	Team exists.	Team record is updated and returned.
UC4	Delete Team	Admin	Remove an incorrect, outdated, or duplicate team entry.	Team exists and can be safely removed.	Team is deleted and removal is confirmed.
UC5	Manage Players (CRUD)	Analyst / Admin	Add, retrieve, update, or delete player records linked to teams.	Team associated with the player exists.	Player record is created/updated/removed successfully.
UC6	Manage Matches (CRUD)	Analyst / Admin	Create, retrieve, update, or delete match records involving two teams.	Home and away teams exist.	Match data stored or updated; invalid team IDs rejected.
UC7	Generate Leaderboards	Analyst / Client App	Produce a ranked leaderboard based on match performance metrics.	Match data exists.	Leaderboard is returned using defined ranking logic.

UC8	View Performance Summaries	Analyst / Client App	Retrieve summary statistics for a team or player.	Relevant team/player and match records exist.	A performance summary is generated and returned.
UC9	Manage Win Probabilities	Analyst / Client App	View or update a team's probability of winning the competition.	Team exists; probability entry exists or can be created.	Win probability is returned or updated with validation.

Data Model Tables

Team Table

Field Name	Type	Key	Description
Id	INTEGER	PK	Unique identifier for each team
Name	VARCHAR		Team name
Country	VARCHAR		Country the team is from (optional)
Manager	VARCHAR		Manager Name

Players Table

Field Name	Type	Key	Description
id	INTEGER	PK	Unique identifier for each player
team_id	INTEGER	FK > Teams.id	Links the player to their team
name	VARCHAR		Player name
position	VARCHAR		Player position (optional)
appearances	INTEGER		Number of appearances (optional)

Matches Table

Field Name	Type	Key	Description
id	INTEGER	PK	Unique identifier for each match
home_team_id	INTEGER	FK > Teams.id	Home team
away_team_id	INTEGER	FK > Teams.id	Away team
date	DATETIME		Match date
home_score	INTEGER		Goals scored by home team

away_score	INTEGER		Goals scored by away team
------------	---------	--	---------------------------

Results Probabilities Table

Field Name	Type	Key	Description
id	INTEGER	PK	Unique identifier for each probability record
team_id	INTEGER	FK > Teams.id	Team the probability refers to
season	VARCHAR		Season the probability applies to
probability	FLOAT		Win/Draw/Loss probability
as_of	DATETIME		Timestamp for when the probability was recorded

Relationship Summary

Relationship	Type
Team → Players	One-to-Many
Team → Matches (home)	One-to-Many
Team → Matches (away)	One-to-Many
Team → WinProbabilities	One-to-Many
Players → Team	Many-to-One
Matches → Teams	Many-to-One (twice: home + away)
WinProbabilities → Team	Many-to-One

AI Usage

AI Declaration

Generative AI tools were used during the development of this project to assist with aspects of implementation and documentation. AI was primarily used to support coding tasks, including generating example code structures, debugging errors, and suggesting improvements to the API implementation. Additionally, AI was used to assist with drafting and proof reading sections of the written report to enhance clarity.

Chat Log Examples

I'm looking to build an API application that satisfies the following basic requirements with the possibility to extend later: implement at least one data model involving all four elements of CRUD (Create, Read, Update and Delete) functionality linked to a database.² You can base your design on a RESTful (Representational State Transfer) API. However, you can also use a GraphQL-based API if you think this is more suitable for the problem, but you must explain the rationale for using GraphQL.

- includes at least four API endpoints accessible via HTTP requests
- handle user inputs and return appropriate JSON responses. You are permitted to use other formats, provided you justify these in your supporting report document.
- use correct status/error codes, according to industry convention
- be demonstrable via local execution, web hosting (e.g. PythonAnywhere), or a Model Context Protocol (MCP) server

I am looking to use Python.
Please set out an actionable set of steps to carry this out



API Development Steps (Python + FastAPI + SQL)

1. Decide the stack
2. Create your project folder
3. Create a virtual environment and install dependencies
4. Implement the database connection
5. Create at least one data model
6. Define input/output JSON schemas
7. Implement CRUD functions
8. Build the API endpoints
9. Use correct status and error codes
10. Run locally and test via HTTP
11. Add basic quality-of-life improvements
12. Prepare for optional hosting or MCP server
13. Write the supporting report
14. Add optional extension features

I want to base my solution around each team in the Europa Leagues chances of winning the competition, how can I apply this context to the framework you have given

Project Summary: Europa League Winning Chances API (FastAPI + SQL)

Core Concept:

Create a RESTful API that stores and manages data about each team in the UEFA Europa League, specifically their **current probability of winning the tournament**. Each database record represents one team's chance at a specific point in time.

Data Model (TeamChance):

- `id` (primary key)
- `team_name` (string)
- `season` (e.g., "2025-26")
- `win_probability` (float 0–1 or 0–100)
- `source` (e.g., "manual", "bookmaker", "model")
- `as_of` (timestamp)
- `notes` (optional)

CRUD Operations:

- **Create:** Add a new team's win probability
- **Read:** Retrieve all teams or a specific team
- **Update:** Modify a team's probability as matches are played
- **Delete:** Remove or clean up outdated entries

Required API Endpoints (meets the 4-endpoint minimum):

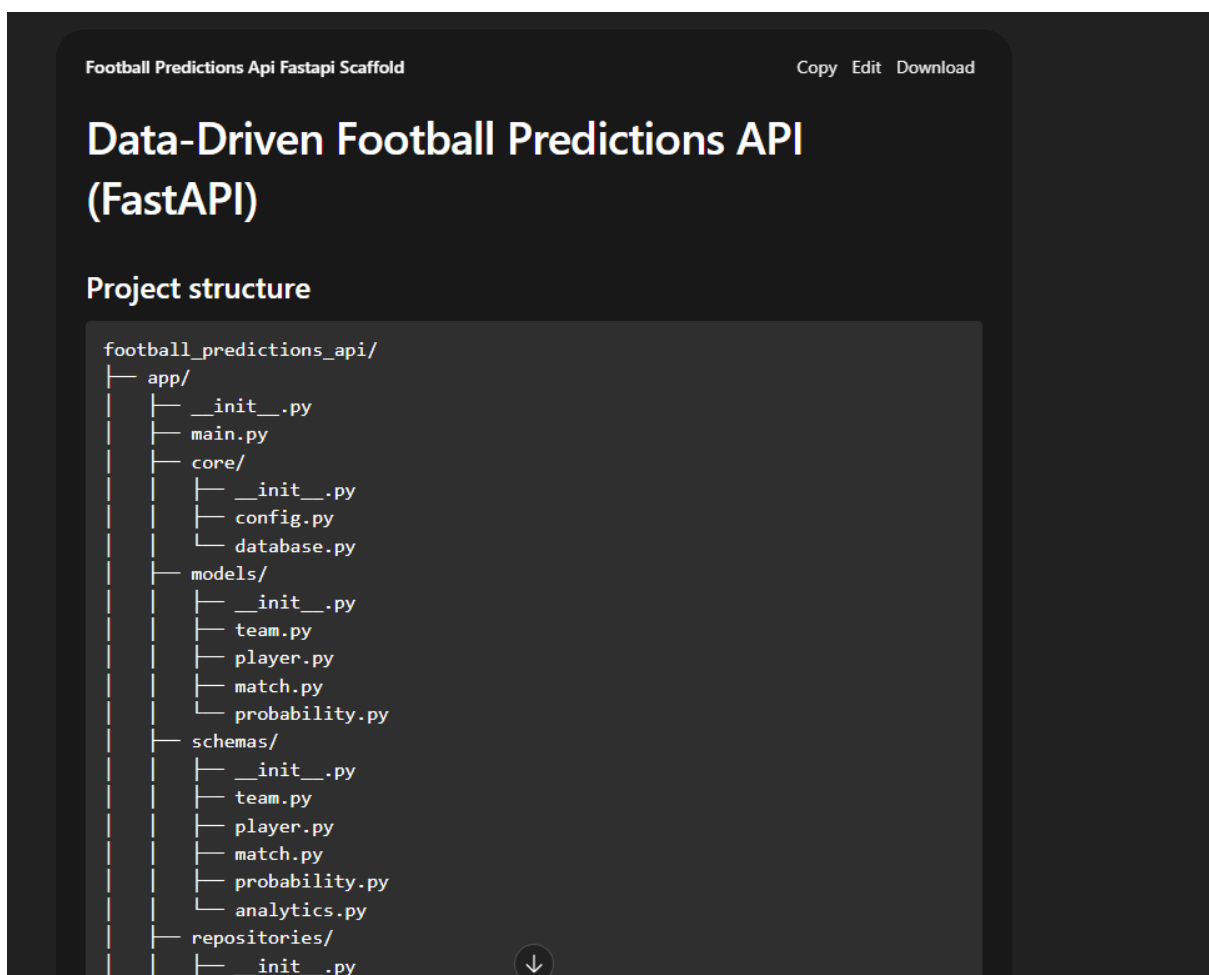
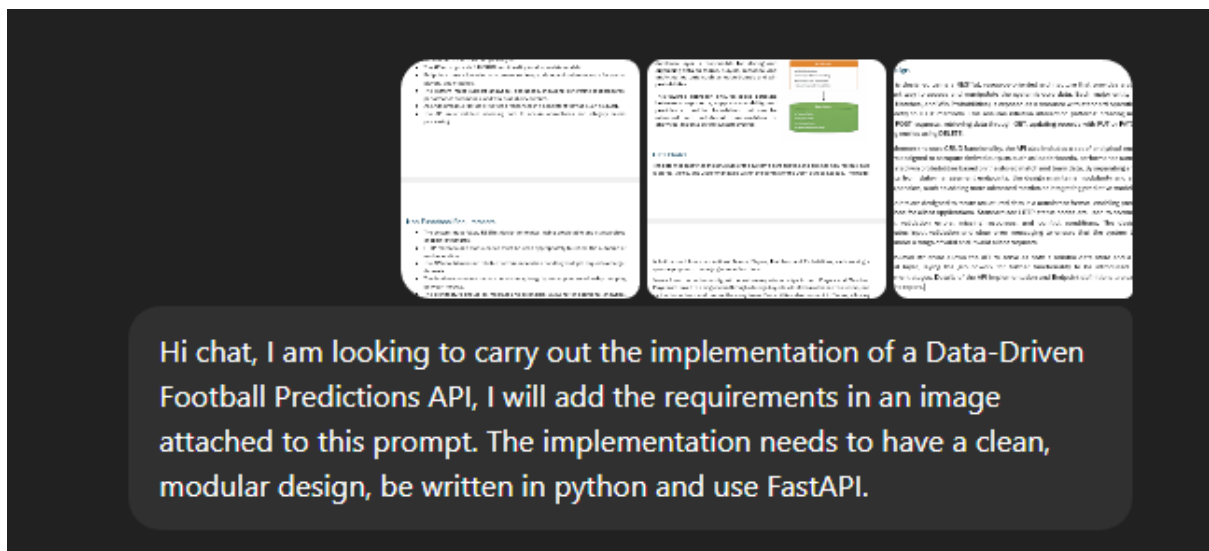
1. `POST /teams` — create a TeamChance record
2. `GET /teams` — list all teams and probabilities
3. `GET /teams/{id}` — retrieve a single team
4. `PATCH /teams/{id}` — update probability
5. `DELETE /teams/{id}` — delete a record (optional but completes CRUD)

Key Features for Coursework Requirements:

- RESTful HTTP design with clear resource naming
- JSON request/response bodies (validatable with Pydantic)
- Proper status codes: 201 (created), 200 (ok), 204 (deleted), 404 (not found), 422 (validation)
- Data stored in an SQL database (SQLite for simplicity)
- Demonstrable via FastAPI's `/docs` interface or local hosting
- Easily extensible later (e.g., model odds over time, add match performance data, integrate external sources)

Why This Context Works Well:

- Shows real-world data modelling
- Naturally supports CRUD
- Easy to justify in the report
- Can be kept simple or expanded into analytics later



This prompt also produced the code/text that would go within each code module. (See commit history on git for full code)

What data do I need? Obviously teams, the potential matches (tournament draw), squad lists, anything else?

You need more than just teams, draw paths, and squads. Think in layers: core competition data, event/state data, team/player performance data, and prediction inputs.

For your API, the minimum useful dataset is:

Core reference data

- teams
- players
- competition
- season
- stage/round
- venues
- managers/coaches
- referees, optionally

Tournament structure

- group allocations or knockout bracket
- draw results
- round definitions
- match pairings
- leg information for home/away ties

Plus many more fields.